

位置变化下的模型归纳偏置 与 ViT Patch 设计对比实验

位置可变 FashionMNIST 分类 · 三组扩展实验报告

研究对象	64×64 画布上的随机位置与固定位置 FashionMNIST
对比内容	模型结构、Patch Size、Patch Embedding
实验协议	验证集选择最佳模型，官方测试集仅作最终评价
实验环境	PyTorch 2.11 · CUDA 12.8 · RTX 5070 Laptop GPU

短学期深度学习课程实验 · 2026 年夏

目录

摘要	3
1 实验目标与设计	4
1.1 研究问题	4
1.2 四种评价设置	4
1.3 统一实验协议	4
1.4 模型配置	5
2 总体结果	6
3 MLP、CNN 与 ViT 对比	7
3.1 实验结果	7
3.2 位置泛化分析	7
4 ViT Patch Size 对比	9
4.1 精度结果	9
4.2 计算成本与折中	9
5 Patch Embedding 实现对比	11
5.1 实验结果	11
5.2 数学关系	11
6 与同学项目的外部参考	12
7 局限性	12
8 结论	13
附录 A 关键代码	14
A.1 实验配置: experiments/comparisons/configs.py	15
A.2 对比模型: experiments/comparisons/models.py	16
A.3 数据划分与协议配置	17
A.4 Checkpoint 选择与训练流程	18
A.5 四种 setting 的最终评价	20
A.6 Patch Embedding 等价性测试	20
附录 B 复现命令	20
参考资料	21

摘要

本实验围绕位置可变 FashionMNIST 分类中的三个扩展问题展开。首先，在统一训练与评价协议下比较 MLP、CNN 与 Vision Transformer (ViT)，考察不同结构归纳偏置对位置分布变化的影响；其次，保持 ViT 主体结构不变，比较 patch size 为 4、8、16 时的准确率与训练成本；最后，对比 Conv2d 与 Flatten+Linear 两种 patch embedding 实现。全部配置使用相同的数据划分、优化器、训练轮数和随机种子，最佳 checkpoint 由验证集选择，官方测试集仅用于最终评价。结果表明，CNN 在四个位置 setting 中均取得最高准确率，尤其在固定位置训练、随机位置测试的 B→A 设置下达到 35.40%；patch size 8 在精度与成本之间表现出更合理的折中；两种 patch embedding 的最大准确率差异仅为 0.61 个百分点，与其数学等价性相符。实验同时显示，所有模型在 B→A 下都存在明显下降，说明只使用固定位置样本训练不足以获得稳定的位置泛化能力。

关键词：FashionMNIST；位置泛化；卷积神经网络；Vision Transformer；Patch Embedding

1 实验目标与设计

1.1 研究问题

基础任务把 28×28 的 FashionMNIST 图像放入 64×64 黑色画布。A 表示图像在画布范围内随机平移，B 表示图像固定在画布中心。本报告不重复基础 ViT 实现，而是集中回答三个对比问题。

- 模型结构：MLP、CNN 与 ViT 谁更能适应位置分布变化？
- Patch Size：更细的 patch 是否必然带来更好的分类结果？
- Patch Embedding：卷积切块与 Flatten+Linear 投影是否产生实质差异？

1.2 四种评价设置

箭头左侧表示训练分布，右侧表示测试分布。A→A 与 B→B 衡量同分布性能，A→B 和 B→A 衡量训练与测试位置分布不一致时的泛化能力。

Setting	训练分布	测试分布	用途
1	A：随机位置	A：随机位置	随机位置同分布评价
2	B：固定居中	B：固定居中	固定位置同分布评价
3	A：随机位置	B：固定居中	随机训练向中心位置泛化
4	B：固定居中	A：随机位置	固定训练向随机位置泛化

1.3 统一实验协议

FashionMNIST 官方训练集按固定随机顺序划分为 54,000 个训练样本和 6,000 个验证样本；官方测试集包含 10,000 个样本。每个配置只训练 A 模型和 B 模型各一次，再分别在 A、B 测试集上评价，从而得到四个 setting。checkpoint 依据同分布验证集准确率选择，避免使用测试集选择最佳 epoch。

项目	设置	项目	设置
画布大小	64×64	训练轮数	15
Batch size	64	初始学习率	1e-03
优化器	AdamW	权重衰减	1e-04
验证集比例	10%	随机种子	42
混合精度	开启	模型选择	最佳验证准确率
Python / PyTorch	3.12.13 / 2.11.0+cu128	GPU	NVIDIA GeForce RTX 5070 Laptop GPU

1.4 模型配置

三组实验共包含六个配置。ViT 均使用 128 维 token、4 层 Transformer Encoder、4 个注意力头、512 维前馈层、GELU 和可学习绝对位置编码。模型结构对比并非严格参数量匹配，结论主要针对当前课程实验配置。

配置	关键结构	Patch	Token 数	参数量
MLP	4096→128→128→10	-	-	542,474
CNN	32/64/128 通道卷积 + 池化	-	-	205,994
ViT (patch 16)	4 层 Encoder, 4 heads	16	16	829,834
ViT (patch 8)	4 层 Encoder, 4 heads	8	64	811,402
ViT (patch 4)	4 层 Encoder, 4 heads	4	256	829,834
ViT (Flatten+Linear)	patch 16, 线性投影	16	16	829,834

2 总体结果

表 3 汇总六个配置在四种 setting 下的最终测试准确率。该表用于统一查阅，后续各节只在对应控制变量范围内进行比较。

配置	参数量	A→A	B→B	A→B	B→A
MLP	542,474	71.89%	89.57%	71.97%	13.99%
CNN	205,994	92.35%	93.14%	92.87%	35.40%
ViT (patch 16)	829,834	80.07%	88.87%	80.97%	16.91%
ViT (patch 8)	811,402	80.40%	88.27%	81.13%	18.81%
ViT (patch 4)	829,834	75.28%	87.22%	74.97%	17.02%
ViT (Flatten+Linear)	829,834	79.67%	88.80%	80.48%	16.30%

表 3 六个配置在四种位置 setting 下的测试准确率

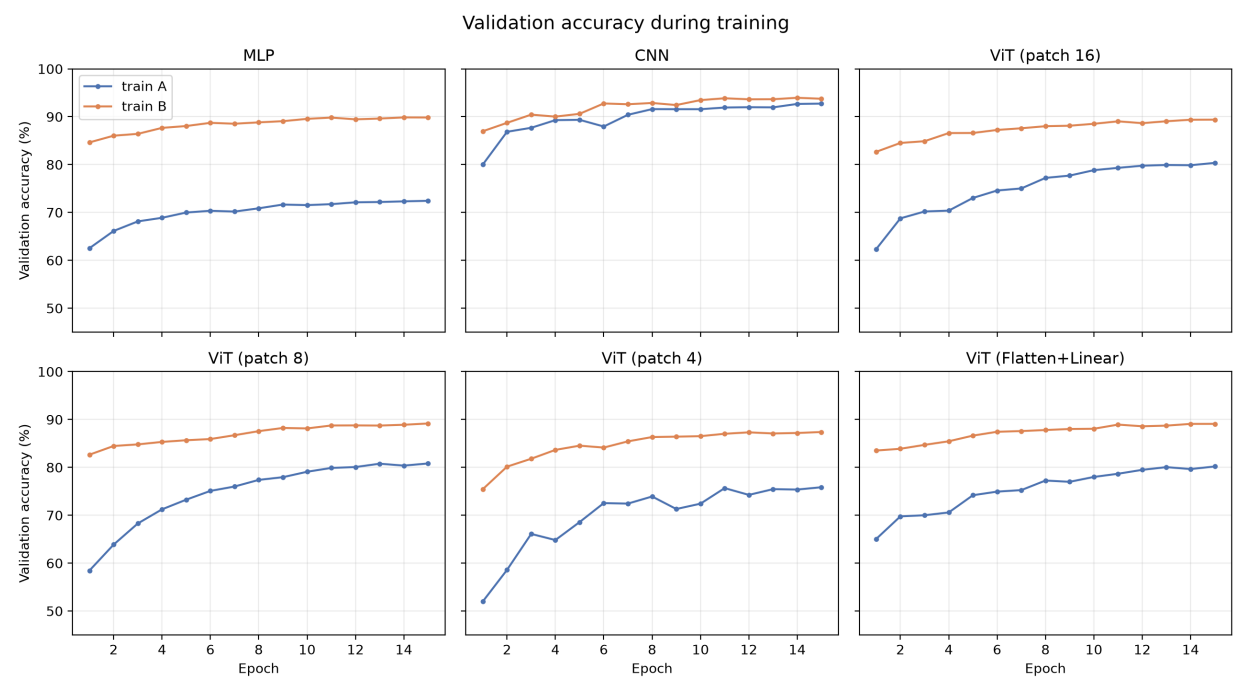


图 1 六种配置在 A、B 训练分布上的验证准确率变化。曲线整体在 15 个 epoch 内趋于稳定，未出现持续发散。

训练曲线显示，B 分布只包含中心位置，任务相对集中，因此多数模型在 B 验证集上的准确率更高、前期上升更快。A 分布包含随机平移，模型需要同时处理类别与位置变化，验证准确率普遍较低。后续分析重点不是比较 A/B 数据集的绝对难度，而是观察同一个训练分布在位置发生变化后能保留多少性能。

3 MLP、CNN 与 ViT 对比

3.1 实验结果

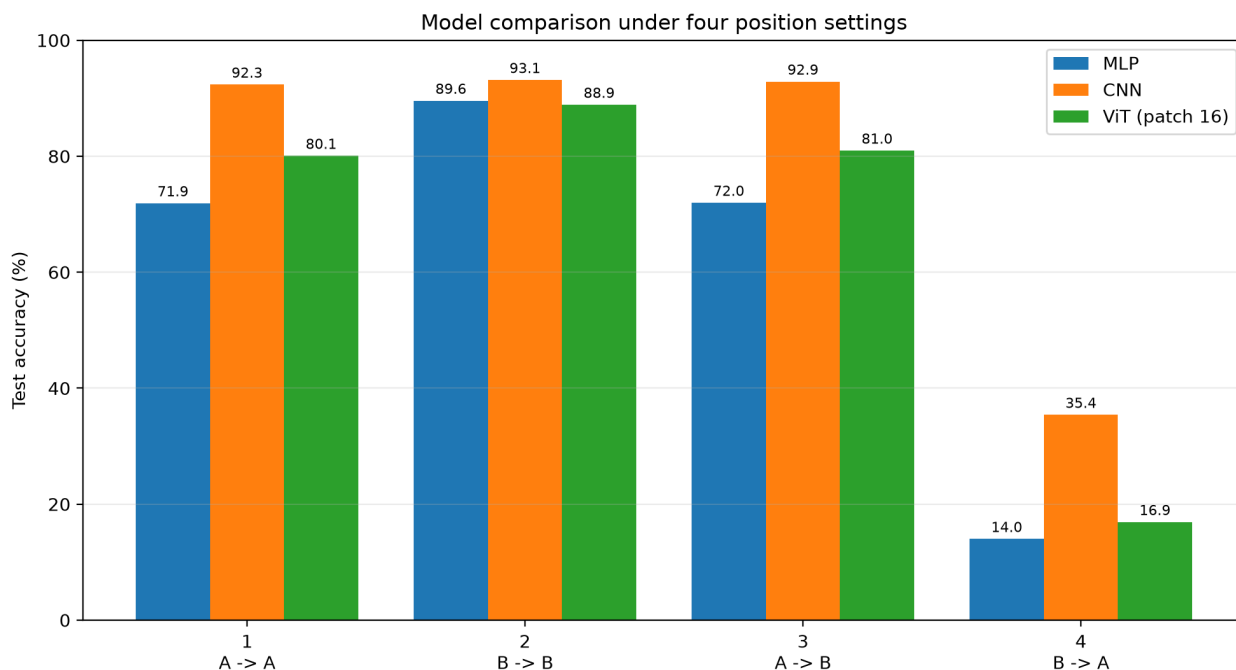


图 2 MLP、CNN 与 patch-16 ViT 在四种位置 setting 下的测试准确率。

CNN 在四个 setting 中均取得最高准确率：A→A 为 92.35%，B→B 为 93.14%，A→B 为 92.87%，B→A 为 35.40%。在同分布或从随机位置训练迁移到中心位置测试时，CNN 都保持在 92% 以上。相比之下，MLP 的 A→A 与 A→B 约为 72%，patch-16 ViT 约为 80% 至 81%。

3.2 位置泛化分析

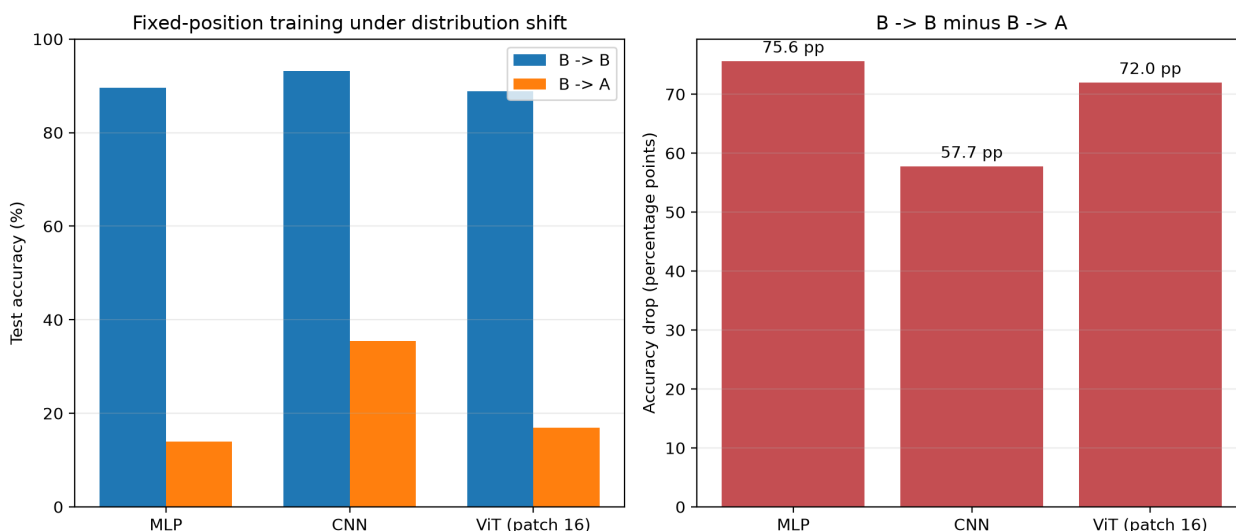


图 3 固定位置训练时的分布变化。右图给出 B→B 与 B→A 的准确率差值。

从 B→B 切换到 B→A 后，MLP、CNN 和 ViT 分别下降 75.58、57.74 和 71.96 个百分点。CNN 的下降幅度最小，B→A 也比 MLP 和 ViT 分别高 21.41 和 18.49 个百分点。

这一现象与模型结构的空间归纳偏置相符。CNN 通过局部感受野和跨位置共享的卷积核重复使用同一特征检测器，因而对平移具有更有利的结构先验；MLP 对每个像素位置使用独立连接，难以把中心位置学到的模式直接迁移到其他位置；当前 ViT 使用可学习绝对位置编码，token 表征与训练位置绑定较强。需要强调的是，CNN 仍下降 57.74 个百分点，因此局部参数共享只能缓解而不能消除由固定位置训练造成的分布偏移。

结论：CNN 是当前配置中最有效的模型基线；随机位置训练对三类模型都很重要，不能用固定居中训练的高准确率替代位置泛化评价。

4 ViT Patch Size 对比

4.1 精度结果

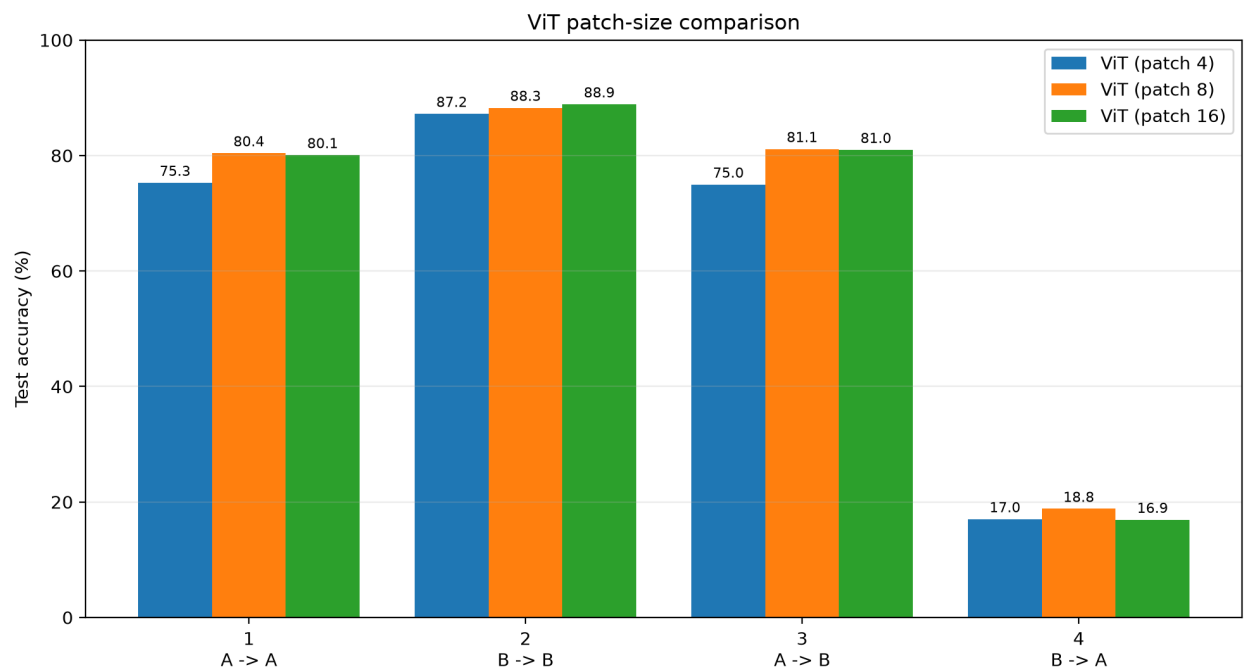


图 4 保持 ViT 主体不变时，patch size 4、8、16 的测试准确率。

patch size 8 在 A→A、A→B 与 B→A 上分别达到 80.40%、81.13% 和 18.81%，均为三个 patch 配置中的最高值；patch size 16 在 B→B 上略高，达到 88.87%。patch size 4 的 A→A 与 A→B 下降到 75.28% 和 74.97%，没有体现出更细分块带来的精度收益。

4.2 计算成本与折中

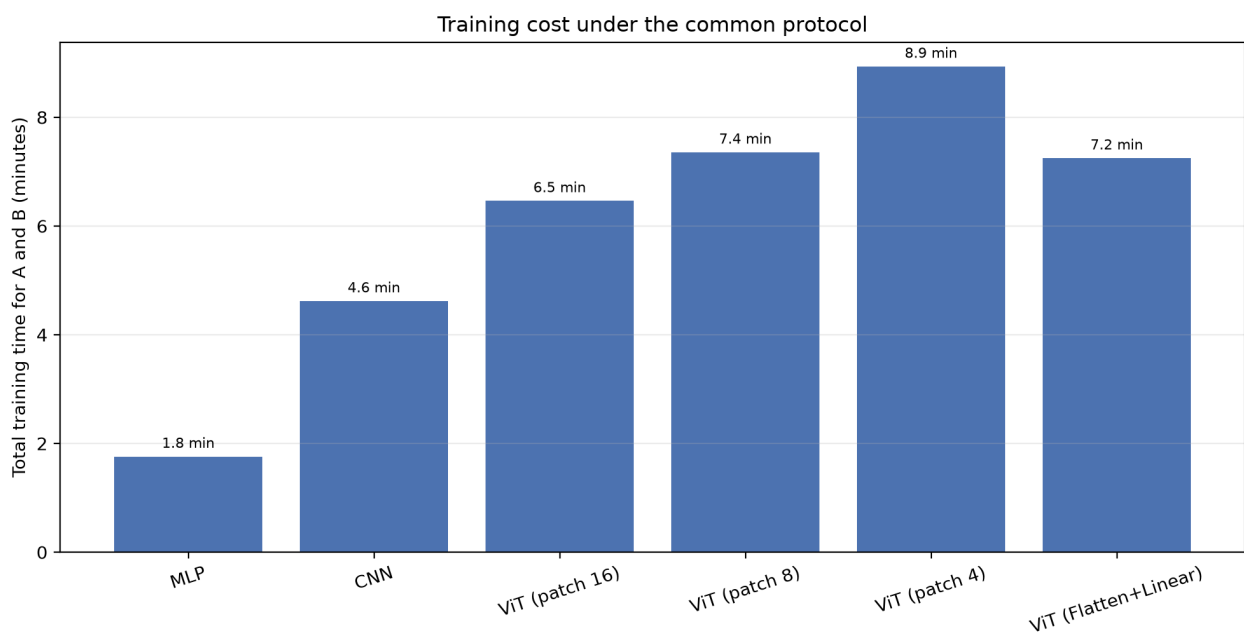


图 5 统一协议下，各配置训练 A、B 两个模型的累计时间。该时间仅反映当前硬件。

Patch size	Token 数	注意力矩阵规模	A/B 总训练时间
4	256	$256^2 = 65,536$	8.93 min
8	64	$64^2 = 4,096$	7.36 min
16	16	$16^2 = 256$	6.46 min

表 4 不同 patch size 的 token 数、理论注意力规模与实测训练时间

在 64×64 画布上，patch size 4、8、16 分别产生 256、64、16 个图像 token。自注意力的两两关系规模随 token 数平方增长，因此 patch size 4 的注意力矩阵元素数在理论上是 patch size 16 的 256 倍。端到端训练时间不会按同样比例增长，因为数据加载、线性层、GPU 并行效率等也占用时间；实际记录中，patch size 4 训练两个模型需要 8.93 分钟，仍是三个设置中最慢的。

更小 patch 提供更细的局部采样，但也增加序列长度和优化难度。在本实验规模下，patch size 8 同时避免了 patch size 16 的粗粒度和 patch size 4 的长序列负担，表现出较合理的精度与成本折中。该结论只适用于当前画布、模型深度和训练预算，不能直接外推到更大数据集或更深 ViT。

结论：更小的 patch 并不必然提高分类准确率；当前配置优先采用 patch size 8。

5 Patch Embedding 实现对比

5.1 实验结果

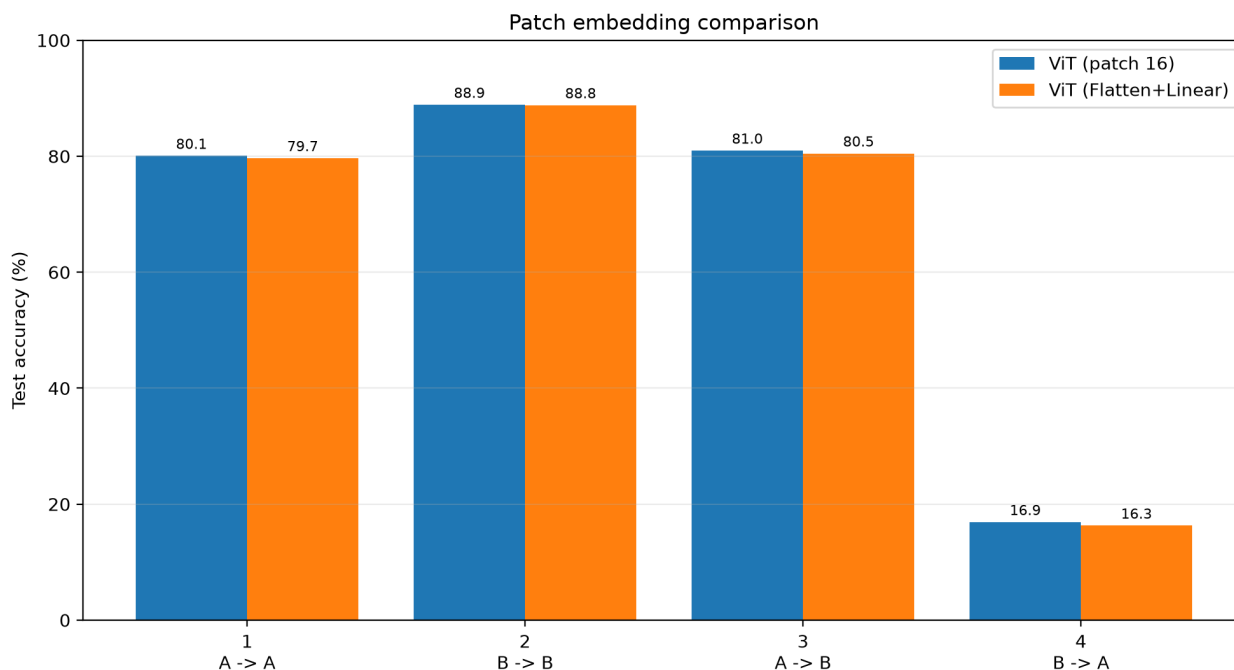


图 6 patch size 16 时，Conv2d 与 Flatten+Linear patch embedding 的结果。

两种实现的参数量均为 829,834。Conv2d 版本在 $A \rightarrow A$ 、 $B \rightarrow B$ 、 $A \rightarrow B$ 、 $B \rightarrow A$ 上分别得到 80.07%、88.87%、80.97%、16.91%；Flatten+Linear 版本分别为 79.67%、88.80%、80.48%、16.30%。四项差异依次为 0.40、0.07、0.49、0.61 个百分点，最大差异为 0.61 个百分点。

5.2 数学关系

当卷积核大小和步长都等于 patch size，且 patch 之间不重叠时，Conv2d 会对每个 patch 使用同一组卷积核权重。把每个卷积核展平后，它对应一个从 patch_dim 到 embed_dim 的线性变换。因此，Conv2d patch embedding 与对每个展平 patch 共享同一个 Linear 层在表达形式上等价。项目测试通过权重重排验证了两种实现的输出在浮点误差范围内一致。

单次训练中仍会出现不足一个百分点的差异，可能来自底层算子、随机训练过程和浮点计算顺序，而不是两种 embedding 具有不同的理论表达能力。当前只有一个随机种子，不能对 0.61 个百分点的差异作显著性判断。

结论：当前结果不支持“卷积版 patch embedding 更强”的判断。在非重叠 patch 设置下，两种实现应主要按照代码可读性和工程便利性选择。

6 与同学项目的外部参考

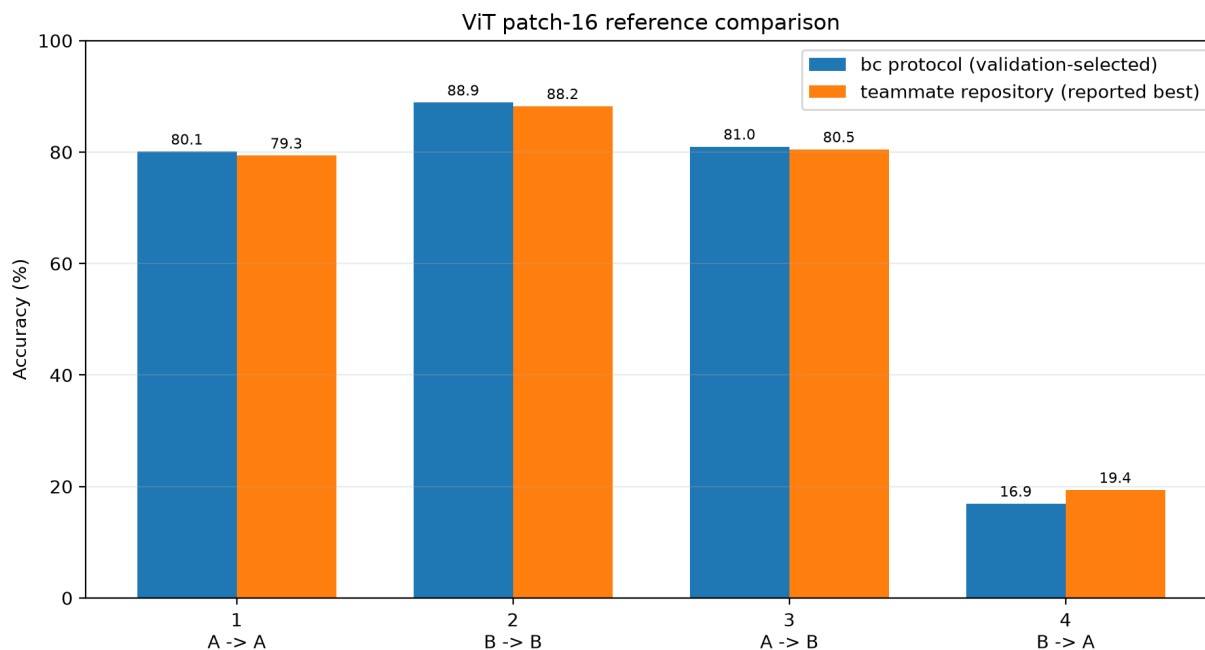


图 7 本项目 patch-16 ViT 与同学仓库公开记录的参考比较。

外部数据来自 kicious/translated-fashion-mnist-vit 仓库 commit

943fa7b68730bc8ea7786bb41c7b8dc1d488883a。该版本公开记录的四项最佳准确率为 79.35%、88.22%、80.48% 和 19.37%；本项目对应结果为 80.07%、88.87%、80.97% 和 16.91%。前三项相差不足一个百分点，B→A 相差 2.46 个百分点。

这组数字不能解释为严格的模型优劣比较。同学仓库在每个 epoch 都使用测试集评价并保留 reported best，本项目则使用独立验证集选择 checkpoint，测试集只评价一次。由于模型选择规则和训练实现不同，图 7 的作用是检查结果量级是否一致，而不是进行统计等价比较。

7 局限性

- 正式结果来自 seed 42 的一次训练，没有重复多个随机种子，因此不报告均值、标准差或显著性。
- CUDA 训练未开启完全确定性；小于一个百分点的差异应视为观察值，不能作过强解释。
- MLP、CNN 与 ViT 的参数量并未严格匹配，模型对比同时包含结构和容量差异。
- 训练时间只来自 RTX 5070 Laptop GPU 的一次记录，不能作为跨硬件基准。
- 实验只覆盖 FashionMNIST、64×64 画布和一种平移规则，结论不直接外推到自然图像、缩放、旋转或更复杂背景。
- 同学项目结果与本项目的 checkpoint 选择规则不同，只能作为外部参考。

8 结论

本报告在统一协议下完成了三组对比。第一，CNN 在所有 setting 上均取得最高准确率，并在 B→A 中显示出相对更强的位置泛化能力，但固定位置训练造成的下降依然明显。第二，ViT 的 patch size 8 在当前训练预算下取得了较好的精度与计算成本平衡，patch size 4 增加了序列长度和训练时间，却没有获得精度收益。第三，Conv2d 与 Flatten+Linear patch embedding 的结果非常接近，结合数学等价性和单元测试，没有证据表明其中一种实现具有稳定优势。

三组实验共同说明，位置泛化不仅取决于模型容量，也与空间参数共享、训练位置覆盖范围和 token 化方式有关。对当前任务而言，直接增加 token 数不如选择合适的结构归纳偏置和训练分布更有效。

附录 A 关键代码

附录列出三组对比直接依赖的配置、模型、实验协议和等价性测试。图表生成、通用工具及基础数据集代码不重复附录，完整代码见仓库对应目录。

A.1 实验配置: experiments/comparisons/configs.py

```

"""Experiment definitions shared by the runner and visualizations."""

from __future__ import annotations

from dataclasses import dataclass

@dataclass(frozen=True)
class ExperimentDefinition:
    config_id: str
    display_name: str
    model_type: str
    patch_size: int | None = None
    patch_embedding: str | None = None

EXPERIMENTS: dict[str, ExperimentDefinition] = {
    "mlp": ExperimentDefinition(
        config_id="mlp",
        display_name="MLP",
        model_type="mlp",
    ),
    "cnn": ExperimentDefinition(
        config_id="cnn",
        display_name="CNN",
        model_type="cnn",
    ),
    "vit_p16_conv": ExperimentDefinition(
        config_id="vit_p16_conv",
        display_name="ViT (patch 16)",
        model_type="vit",
        patch_size=16,
        patch_embedding="conv",
    ),
    "vit_p8_conv": ExperimentDefinition(
        config_id="vit_p8_conv",
        display_name="ViT (patch 8)",
        model_type="vit",
        patch_size=8,
        patch_embedding="conv",
    ),
    "vit_p4_conv": ExperimentDefinition(
        config_id="vit_p4_conv",
        display_name="ViT (patch 4)",
        model_type="vit",
        patch_size=4,
        patch_embedding="conv",
    ),
    "vit_p16_linear": ExperimentDefinition(
        config_id="vit_p16_linear",
        display_name="ViT (Flatten+Linear)",
        model_type="vit",
        patch_size=16,
        patch_embedding="linear",
    ),
}

GROUPS: dict[str, tuple[str, ...]] = {
    "model": ("mlp", "cnn", "vit_p16_conv"),
    "patch_size": ("vit_p4_conv", "vit_p8_conv", "vit_p16_conv"),
    "patch_embedding": ("vit_p16_conv", "vit_p16_linear"),
}

SETTING_ORDER: tuple[tuple[int, str, str], ...] = (
    (1, "A", "A"),
    (2, "B", "B"),
    (3, "A", "B"),
    (4, "B", "A"),
)

def configuration_ids_for_groups(groups: list[str]) -> list[str]:
    selected: set[str] = set()
    if "all" in groups:
        groups = list(GROUPS)
    for group in groups:
        selected.update(GROUPS[group])
    return [config_id for config_id in EXPERIMENTS if config_id in selected]

def groups_for_configuration(config_id: str) -> list[str]:
    return [group for group, members in GROUPS.items() if config_id in members]

```

A.2 对比模型：experiments/comparisons/models.py

```

"""MLP, CNN and ViT models used in the comparison study."""

from __future__ import annotations

from torch import nn

from models import VisionTransformer

from .configs import ExperimentDefinition

class MLPClassifier(nn.Module):
    """Parameter-scale-matched MLP for 64x64 grayscale inputs."""

    def __init__(self, image_size: int = 64, num_classes: int = 10) -> None:
        super().__init__()
        input_dim = image_size * image_size
        hidden_dim = 128
        self.network = nn.Sequential(
            nn.Flatten(),
            nn.Linear(input_dim, hidden_dim),
            nn.LayerNorm(hidden_dim),
            nn.GELU(),
            nn.Dropout(0.1),
            nn.Linear(hidden_dim, hidden_dim),
            nn.GELU(),
            nn.Dropout(0.1),
            nn.Linear(hidden_dim, num_classes),
        )

    def forward(self, images):
        return self.network(images)

class CNNClassifier(nn.Module):
    """Compact CNN with local receptive fields and shared spatial filters."""

    def __init__(self, num_classes: int = 10) -> None:
        super().__init__()
        self.features = nn.Sequential(
            nn.Conv2d(1, 32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.GELU(),
            nn.Conv2d(32, 32, kernel_size=3, padding=1),
            nn.GELU(),
            nn.MaxPool2d(2),
            nn.Conv2d(32, 64, kernel_size=3, padding=1),
            nn.BatchNorm2d(64),
            nn.GELU(),
            nn.Conv2d(64, 64, kernel_size=3, padding=1),
            nn.GELU(),
            nn.MaxPool2d(2),
            nn.Conv2d(64, 128, kernel_size=3, padding=1),
            nn.GELU(),
            nn.AdaptiveAvgPool2d((2, 2)),
        )
        self.classifier = nn.Sequential(
            nn.Flatten(),
            nn.Linear(128 * 2 * 2, 128),
            nn.GELU(),
            nn.Dropout(0.1),
            nn.Linear(128, num_classes),
        )

    def forward(self, images):
        return self.classifier(self.features(images))

def build_model(
    definition: ExperimentDefinition,
    image_size: int = 64,
    num_classes: int = 10,
) -> nn.Module:
    if definition.model_type == "mlp":
        return MLPClassifier(image_size=image_size, num_classes=num_classes)
    if definition.model_type == "cnn":
        return CNNClassifier(num_classes=num_classes)
    if definition.model_type == "vit":
        if definition.patch_size is None or definition.patch_embedding is None:
            raise ValueError("ViT definitions require patch size and embedding type.")
        return VisionTransformer(
            image_size=image_size,
            patch_size=definition.patch_size,
            in_channels=1,
            num_classes=num_classes,
            embed_dim=128,

```



```

        depth=4,
        num_heads=4,
        mlp_dim=512,
        dropout=0.1,
        patch_embedding=definition.patch_embedding,
    )
    raise ValueError(f"Unsupported model type: {definition.model_type}")

def count_trainable_parameters(model: nn.Module) -> int:
    return sum(parameter.numel() for parameter in model.parameters() if
                parameter.requires_grad)

```

A.3 数据划分与协议配置

```

@dataclass
class ProtocolConfig:
    data_dir: str = "data"
    output_dir: str = "results/comparisons"
    canvas_size: int = 64
    epochs: int = 15
    batch_size: int = 64
    learning_rate: float = 1e-3
    weight_decay: float = 1e-4
    val_fraction: float = 0.1
    num_workers: int = 4
    seed: int = 42
    device: str = "auto"
    amp: bool = True
    deterministic: bool = False
    download: bool = False
    limit_train_samples: int = 0
    limit_val_samples: int = 0
    limit_test_samples: int = 0

def _limit_dataset(dataset: Dataset, limit: int) -> Dataset:
    if limit <= 0 or limit >= len(dataset):
        return dataset
    return Subset(dataset, range(limit))

def _base_datasets(config: ProtocolConfig):
    transform = transforms.ToTensor()
    train = tv_datasets.FashionMNIST(
        root=config.data_dir,
        train=True,
        download=config.download,
        transform=transform,
    )
    test = tv_datasets.FashionMNIST(
        root=config.data_dir,
        train=False,
        download=config.download,
        transform=transform,
    )
    return train, test

def _split_train_validation(base_train: Dataset, config: ProtocolConfig):
    val_size = round(len(base_train) * config.val_fraction)
    generator = torch.Generator().manual_seed(config.seed)
    order = torch.randperm(len(base_train), generator=generator).tolist()
    val_indices = order[:val_size]
    train_indices = order[val_size:]
    train_subset = _limit_dataset(Subset(base_train, train_indices),
                                    config.limit_train_samples)
    val_subset = _limit_dataset(Subset(base_train, val_indices), config.limit_val_samples)
    return train_subset, val_subset

```

A.4 Checkpoint 选择与训练流程

```
def _can_resume(run_dir: Path, expected_metadata: dict) -> bool:
    metadata_path = run_dir / "metadata.json"
    checkpoint_path = run_dir / "best.pt"
    if not metadata_path.exists() or not checkpoint_path.exists():
        return False
    try:
        existing = json.loads(metadata_path.read_text(encoding="utf-8"))
    except (OSError, json.JSONDecodeError):
        return False
    existing_experiment = existing.get("experiment", {})
    existing_protocol = dict(existing_experiment.get("protocol", {}))
    expected_protocol = dict(expected_metadata.get("protocol", {}))
    existing_protocol.pop("output_dir", None)
    expected_protocol.pop("output_dir", None)
    existing_experiment = {
        **existing_experiment,
        "protocol": existing_protocol,
    }
    portable_expected = {
        **expected_metadata,
        "protocol": expected_protocol,
    }
    return existing_experiment == portable_expected

def fit_model(
    definition: ExperimentDefinition,
    train_mode: str,
    config: ProtocolConfig,
    resume: bool,
):
    set_seed(config.seed, deterministic=config.deterministic)
    device = resolve_device(config.device)
    run_dir = Path(config.output_dir) / "runs" / definition.config_id / f"train_{train_mode}"
    run_dir.mkdir(parents=True, exist_ok=True)
    expected_metadata = _expected_metadata(definition, config, train_mode)
    model = build_model(definition, image_size=config.canvas_size).to(device)
    criterion = nn.CrossEntropyLoss()
    checkpoint_path = run_dir / "best.pt"

    if resume and _can_resume(run_dir, expected_metadata):
        checkpoint = torch.load(
            checkpoint_path,
            map_location=device,
            weights_only=False,
        )
        model.load_state_dict(checkpoint["model_state"])
        result = json.loads((run_dir / "training_result.json").read_text(encoding="utf-8"))
        result["checkpoint"] = str(checkpoint_path)
        save_json(result, run_dir / "training_result.json")
        metadata = json.loads((run_dir / "metadata.json").read_text(encoding="utf-8"))
        metadata["experiment"] = expected_metadata
        save_json(metadata, run_dir / "metadata.json")
        print(f"Reusing {definition.config_id}, train {train_mode}")
        return model, device, criterion, result

    train_loader, val_loader = create_train_validation_loaders(
        config,
        train_mode,
        device,
    )
    optimizer = torch.optim.AdamW(
        model.parameters(),
        lr=config.learning_rate,
        weight_decay=config.weight_decay,
    )
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer,
        T_max=max(config.epochs, 1),
    )
    amp_enabled = bool(config.amp and device.type == "cuda")
    scaler = torch.amp.GradScaler("cuda", enabled=amp_enabled)
    parameter_count = count_trainable_parameters(model)
    save_json(
        {
            "experiment": expected_metadata,
            "parameter_count": parameter_count,
            "device": str(device),
            "torch_version": torch.__version__,
            "train_samples": len(train_loader.dataset),
            "validation_samples": len(val_loader.dataset),
        },
        run_dir / "metadata.json",
    )

    history: list[dict[str, float]] = []
```

```

best_accuracy = -1.0
best_epoch = 0
started = time.time()
print(
    f"Training {definition.config_id}, mode {train_mode} | "
    f"{parameter_count:,} parameters | {device}"
)
for epoch in range(1, config.epochs + 1):
    train_metrics = _train_epoch(
        model,
        train_loader,
        criterion,
        optimizer,
        scaler,
        device,
        amp_enabled,
    )
    val_metrics = evaluate(model, val_loader, criterion, device)
    scheduler.step()
    row = {
        "epoch": epoch,
        "learning_rate": optimizer.param_groups[0]["lr"],
        "train_loss": train_metrics["loss"],
        "train_accuracy": train_metrics["accuracy"],
        "val_loss": val_metrics["loss"],
        "val_accuracy": val_metrics["accuracy"],
    }
    history.append(row)
    print(
        f" epoch {epoch:02d}/{config.epochs} | "
        f"train {100 * row['train_accuracy']:.2f}% | "
        f"val {100 * row['val_accuracy']:.2f}%"
    )
    if row["val_accuracy"] > best_accuracy:
        best_accuracy = row["val_accuracy"]
        best_epoch = epoch
        torch.save(
            {
                "model_state": model.state_dict(),
                "optimizer_state": optimizer.state_dict(),
                "epoch": epoch,
                "best_val_accuracy": best_accuracy,
                "definition": asdict(definition),
            },
            checkpoint_path,
        )

write_csv(history, run_dir / "history.csv")
plot_history(history, run_dir / "training_curves.png")
checkpoint = torch.load(checkpoint_path, map_location=device, weights_only=False)
model.load_state_dict(checkpoint["model_state"])
result = {
    "config_id": definition.config_id,
    "train_mode": train_mode,
    "best_epoch": best_epoch,
    "best_val_accuracy": best_accuracy,
    "elapsed_seconds": time.time() - started,
    "parameter_count": parameter_count,
    "checkpoint": str(checkpoint_path),
}
save_json(result, run_dir / "training_result.json")
return model, device, criterion, result

```

A.5 四种 setting 的最终评价

```
def run_configuration(
    config_id: str,
    protocol: ProtocolConfig,
    resume: bool = True,
) -> list[dict]:
    definition = EXPERIMENTS[config_id]
    measured: dict[tuple[str, str], dict] = {}
    for train_mode in ("A", "B"):
        model, device, criterion, training_result = fit_model(
            definition,
            train_mode,
            protocol,
            resume,
        )
        for test_mode in ("A", "B"):
            test_loader = create_test_loader(protocol, test_mode, device)
            metrics = evaluate(model, test_loader, criterion, device)
            measured[(train_mode, test_mode)] = {
                "config_id": config_id,
                "display_name": definition.display_name,
                "groups": ";".join(groups_for_configuration(config_id)),
                "model_type": definition.model_type,
                "patch_size": definition.patch_size or "",
                "patch_embedding": definition.patch_embedding or "",
                "train_mode": train_mode,
                "test_mode": test_mode,
                "test_loss": metrics["loss"],
                "test_accuracy": metrics["accuracy"],
                "best_val_accuracy": training_result["best_val_accuracy"],
                "best_epoch": training_result["best_epoch"],
                "train_elapsed_seconds": training_result["elapsed_seconds"],
                "parameter_count": training_result["parameter_count"],
            }
            print(
                f" evaluate {train_mode}->{test_mode}: "
                f"{100 * metrics['accuracy']:.2f}%"
            )
    del model
    if torch.cuda.is_available():
        torch.cuda.empty_cache()

    rows: list[dict] = []
    for setting, train_mode, test_mode in SETTING_ORDER:
        rows.append(
            {
                "setting": setting,
                **measured[(train_mode, test_mode)],
            }
        )
    return rows
```

A.6 Patch Embedding 等价性测试

```
def test_conv_and_linear_patch_projection_are_equivalent(self):
    torch.manual_seed(7)
    conv = ConvPatchEmbedding(64, 16, 1, 128)
    torch.manual_seed(7)
    linear = LinearPatchEmbedding(64, 16, 1, 128)
    images = torch.rand(2, 1, 64, 64)
    self.assertTrue(
        torch.allclose(conv(images), linear(images), atol=1e-6, rtol=1e-5)
    )
```

附录 B 复现命令

```
cd /home/bccc/dl-course
source ~/miniconda3/etc/profile.d/conda.sh
conda activate torch101

python -m unittest discover -s tests -v

python -m experiments.comparisons.run \
    --groups all \
    --epochs 15 \
    --batch-size 64 \
    --num-workers 4 \
    --output-dir results/comparisons
```

参考资料

- [1] 课程材料：《关于作业与实验报告》《实验实现》，2026 年夏。
- [2] Xiao H., Rasul K., Vollgraf R. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms, 2017.
- [3] Dosovitskiy A. et al. An Image is Worth 16×16 Words: Transformers for Image Recognition at Scale, ICLR 2021.
- [4] kicious/translated-fashion-mnist-vit, commit 943fa7b68730bc8ea7786bb41c7b8dc1d488883a.